

Desenvolvimento Web 2

Aula 8

Single Page Applications com React

`l.bertholdo@ifsp.edu.br`

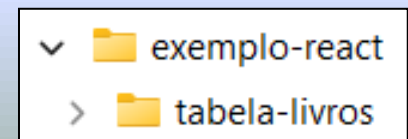
Conteúdo

- Criando uma Aplicação React
- Estrutura de uma Aplicação React
- Exemplo – Tabela de Livros

Criando uma Aplicação React

- O React possui bibliotecas e módulos auxiliares, como o **NodeJS** e o **NPM (*Node Package Manager*)**, que permitem estruturar as pastas, criar uma página inicial padrão customizável, bem como rodar um servidor local para renderizar a aplicação com apenas dois comandos.
- O ponto de partida para criar uma aplicação é o comando **create-react-app nome-aplicacao**, sendo **nome-aplicacao** a pasta raiz da aplicação.
- Criando uma nova aplicação React:
 1. Crie uma pasta denominada **exemplo-react** para armazenar localmente a aplicação **tabela-livros** a ser desenvolvida.
 2. Abra o *prompt* de comando do sistema operacional (como administrador) na pasta criada e execute o comando:

```
npx create-react-app tabela-livros
```



Criando uma Aplicação React

- Criando uma nova aplicação React (continuação):

```
C:\Users\leodb\Documents\VSCodeProjects\exemplo-react>npx create-react-app tabela-livros
Creating a new React app in C:\Users\leodb\Documents\VSCodeProjects\exemplo-react\tabela-livros.
```

*Esse comando demora para ser executado, mas quando finalizado cria uma **estrutura padrão** para aplicações React em uma pasta denominada **tabela-livros**. Ao final da execução, será mostrada a mensagem ao lado.*

We suggest that you begin by typing:

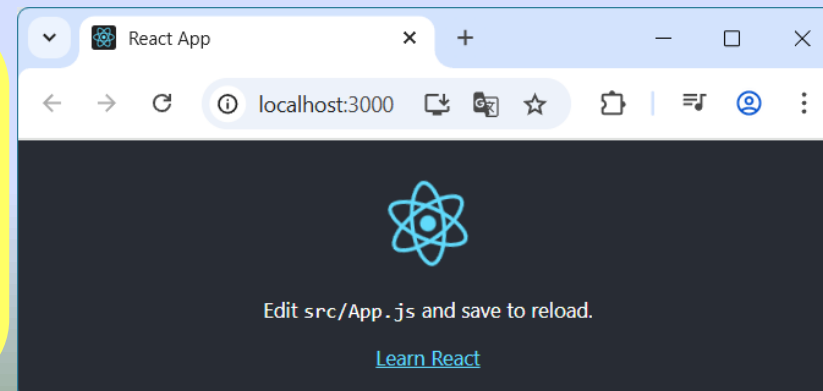
```
cd tabela-livros
npm start
```

Happy hacking!

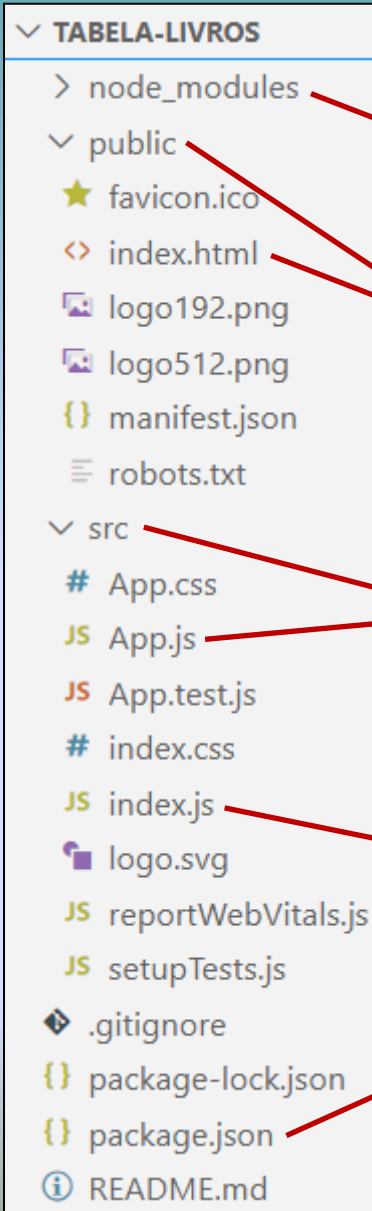
3. Em seguida, execute os dois comandos sugeridos:

```
C:\Users\leodb\Documents\VSCodeProjects\exemplo-react>cd tabela-livros
C:\Users\leodb\Documents\VSCodeProjects\exemplo-react\tabela-livros>npm start
```

*O comando **npm start** inicia o servidor local na porta 3000 (localhost:3000). O navegador padrão é automaticamente aberto e exibe a **página padrão** da aplicação pronta para ser personalizada. Se o navegador não abrir automaticamente, abra-o e acesse **localhost:3000**. Para parar o servidor, acione as teclas **Ctrl+C** no prompt de comando.*



Estrutura de uma Aplicação React



Estrutura padrão de pastas e arquivos da aplicação
React criada com o comando create-react-app

Contém todos os módulos e funcionalidades necessárias do React, além de bibliotecas auxiliares que podem ser instaladas para uma aplicação específica.

Contém o arquivo **index.html** cuja marcação define o container da aplicação localizado na linha `<div id="root"></div>`.

Contém os arquivos nos quais o desenvolvedor mais costuma trabalhar. O arquivo **App.js** define o componente-pai, com toda a marcação HTML da aplicação a ser injetada dentro do elemento `<div id="root">`.

Contém o script que injeta/renderiza o componente **App** no elemento `<div id="root">` do arquivo **index.html**.

Contém um **objeto JSON** que descreve a configuração inicial necessária para a página padrão da aplicação funcionar, além dos nomes de todas as dependências, módulos e suas versões, necessários para fazer a aplicação funcionar.

Estrutura de uma Aplicação React

- Os arquivos **App.css** e **index.css** contêm as regras de estilo CSS da aplicação. Para testá-las, no Visual Studio Code, faça as alterações a seguir, salve e observe o resultado no navegador.

```
src
# App.css
JS App.js
JS App.test.js
# index.css
```

App.css

```
.App-header {
  background-color: ■ red; /* Alterar */
  min-height: 100vh;
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
  font-size: 50px; /* Alterar */
  color: □ white;
}
```

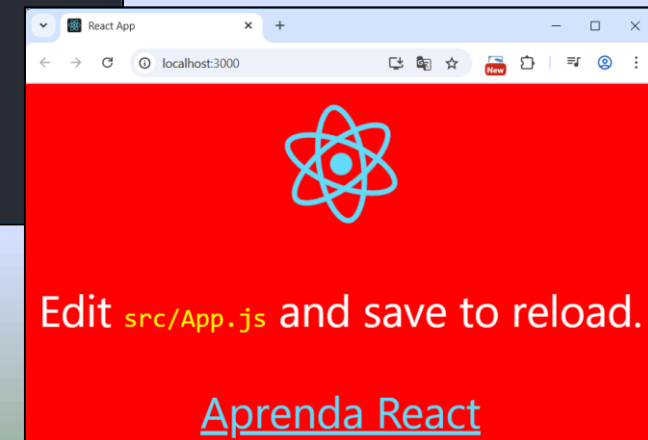
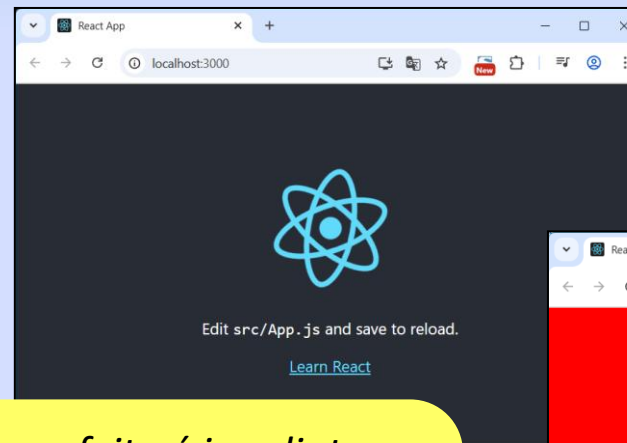
index.css

```
code {
  /* ... */
  → font-size: 30px; /* Incluir */
  → color: ■ yellow; /* Incluir */
}
```

App.js

```
Aprenda React
</a>
</header>
</div>
);
}
```

Altere também o texto do link no arquivo **App.js**.



Assim que as modificações são salvas, o efeito é imediato na página, sem necessidade de recarregá-la. Trata-se de uma das funcionalidades de React: atualizar a página somente no local em que houve a alteração sem ter de recarregar tudo.

Exemplo – Tabela de Livros

- O projeto de exemplo é uma **tabela de livros** na qual constam alguns dados sobre livros, um **botão para excluir** cada livro, **setas para ordenar** as linhas da tabela pelo título do livro, além de um **campo para totalizar** os livros exibidos.

Tabela de Livros			
ISBN	Título  	Autor	
9788535909555	A Revolução dos Bichos	George Orwell	Excluir
9788525056009	Admirável Mundo Novo	Aldous Huxley	Excluir
9786589711612	Crime e Castigo	Fiódor Dostoiévski	Excluir
Quantidade de Livros: 3			

Editando Arquivos

- O ponto de partida do projeto é a estrutura criada anteriormente. Portanto, o primeiro passo é editar alguns arquivos que serão usados na aplicação.

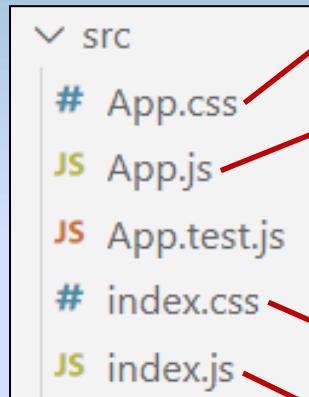
index.html

```
<html lang="pt-br">
<head>
  <meta charset="utf-8" />
  <link rel="icon" href="%PUBLIC_URL%/favicon.ico" />
  <meta name="viewport" content="width=device-width, initial-scale=1" />
  <meta name="theme-color" content="#ffffff" />
  <meta
    name="description"
    content="Projeto Tabela de Livros"
  />
  <link rel="apple-touch-icon" href="%PUBLIC_URL%/logo192.png" />
  <link rel="manifest" href="%PUBLIC_URL%/manifest.json" />
  <title>App React | Tabela de Livros</title>
</head>
<body>
  <noscript>Habilite o JavaScript para visualizar esta aplicação.</noscript>
  <div id="root"></div>
</body>
</html>
```

Remova todos os comentários e faça as alterações indicadas.

A tag **<noscript>** define um conteúdo alternativo a ser exibido em navegadores em que a execução de scripts está desabilitada ou indisponível.

Editando Arquivos



Apague este arquivo.

Remova essas linhas:

```
import logo from './logo.svg';
import './App.css';
```

Substitua o conteúdo do `return` por este:

```
return (
  <div>
    <h1>Tabela de Livros</h1>
  </div>
);
```

Remova todo o conteúdo.

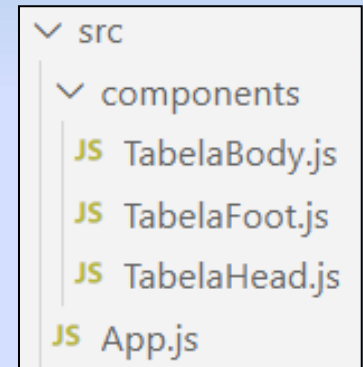
Remova todos os comentários.

Após as modificações, a aplicação será exibida sem qualquer estilização e com o texto "Tabela de Livros" do elemento `<h1>` em um fundo branco.



Criando a Tabela

- A tabela será constituída por um **topo** contendo o título “Tabela de Livros”, um **corpo** com quatro colunas contendo os dados dos livros e um **rodapé** contendo a quantidade de livros da tabela.
- Para isso, serão criados três componentes, um para cada seção da tabela. Esse é um padrão elementar ao usar React: criar **pequenos componentes** para formar uma peça da interface.
- Na pasta **src** do projeto, crie a subpasta **components**. Na subpasta criada, defina um arquivo JavaScript para cada componente e implemente seus códigos.
- O componente **App** é o container geral da aplicação e responsável por injetar toda a marcação HTML no DOM. Por isso, também será preciso inserir os três componentes da tabela dentro do componente-pai **App**, o qual é definido no arquivo **App.js**.



Criando a Tabela

```
import React from "react";
const TabelaHead = () => (
  <thead>
    <tr>
      <th colspan="4">Tabela de Livros</th>
    </tr>
    <tr>
      <th>ISBN</th>
      <th>Título</th>
      <th>Autor</th>
      <th></th>
    </tr>
  </thead>
);
export default TabelaHead;
```

TabelaHead.js

```
import React from "react";
const TabelaFoot = () => (
  <tfoot>
    <tr>
      <td colspan="4">Rodapé da Tabela</td>
    </tr>
  </tfoot>
);
export default TabelaFoot;
```

TabelaFoot.js

```
import React from "react";
const TabelaBody = () => (
  <tbody>
    <tr>
      <td>9786589711612</td>
      <td>Crime e Castigo</td>
      <td>Fiódor Dostoiévski</td>
      <td>
        <button className="excluir">Excluir</button>
      </td>
    </tr>
  </tbody>
);
export default TabelaBody;
```

TabelaBody.js

```
import React from "react";
import TabelaHead from "../components/TabelaHead";
import TabelaFoot from "../components/TabelaFoot";
import TabelaBody from "../components/TabelaBody";
function App() {
  return (
    <table className="tabela">
      <TabelaHead />
      <TabelaFoot />
      <TabelaBody />
    </table>
  );
}
export default App;
```

App.js

Criando a Tabela

- Tabela renderizada

Tabela de Livros			
ISBN	Título	Autor	
9786589711612	Crime e Castigo	Fiódor Dostoiévski	Excluir
Rodapé da Tabela			

Enviando Dados para a Tabela

- Os dados exibidos na única linha da tabela foram inseridos diretamente no código do componente **TabelaBody**. Para deixar este componente flexível, tais dados passarão a ser armazenados no seu componente-pai **App**, que então os enviará ao componente **TabelaBody**.
- Até a versão 16.7, o fluxo de dados do React ocorria em **um só sentido**, com os dados sendo passados do componente-pai para os componentes descendentes. Além disso, também não era possível passar dados com uso de **componentes funcionais**, apenas com **componentes do tipo classe**.
- A partir da versão 16.8, foi criada uma nova funcionalidade no React, chamada **hooks**, que permite passar dados com uso de **componentes funcionais** em **fluxos de dados multidirecionais** (um componente pode passar dados para componentes descendentes, ascendentes e irmãos).

Enviando Dados para a Tabela

- O componente **App** é o container geral da aplicação e, por padrão, é um **componente do tipo funcional**.
- No exemplo, será usada a forma tradicional (sem uso de *hooks*) para passar dados do componente **App** para seus filhos **TabelaBody** e **TabelaFoot**. Para isso, vamos transformá-lo em um **componente do tipo classe**.

```
function App() {  
  return (  

```

App.js

```
import React, { Component } from "react";  
import TabelaHead from "../components/TabelaHead";  
import TabelaFoot from "../components/TabelaFoot";  
import TabelaBody from "../components/TabelaBody";  
  
class App extends Component {  
  render() {  
    return (  
      <table className="tabela">  
        <TabelaHead />  
        <TabelaFoot />  
        <TabelaBody />  
      </table>  
    );  
  }  
}  
export default App;
```

Para trabalhar com classes em React, é preciso importar **Component**, que é a classe base para definir componentes como classes JavaScript.

Sintaxe padrão para declarar uma classe em React:
`class nome_classe extends Component`

Uso do método React **render()** para retornar a tabela.

Enviando Dados para a Tabela

- Para armazenar dados em um componente-pai, o React fornece o objeto **state**.

Armazena os dados de três livros no objeto **state**.

Passa os dados dos três livros para o componente **TabelaBody**. A sintaxe para a passagem dos dados é composta por um **atributo** (livros) e um **valor** (this.state.livros). O nome do **atributo** é de livre escolha e o **valor** deve ser escrito entre chaves.

App.js

```
class App extends Component {
  state = {
    livros: [
      {
        id: "9786589711612",
        titulo: "Crime e Castigo",
        autor: "Fiódor Dostoiévski"
      },
      {
        id: "9788525056009",
        titulo: "Admirável Mundo Novo",
        autor: "Aldous Huxley"
      },
      {
        id: "9788535909555",
        titulo: "A Revolução dos Bichos",
        autor: "George Orwell"
      }
    ]
  };
  render() {
    return (
      <table className="tabela">
        <TabelaHead />
        <TabelaFoot />
        <TabelaBody livros={ this.state.livros } />
      </table>
    );
  }
}
export default App;
```

Enviando Dados para a Tabela

- O componente **TabelaBody** também precisa ser modificado para receber os dados do componente-pai **App**.

Agora, o componente recebe os dados no parâmetro **props** da função que o define.

props.livros, objeto que contém os dados vindos do componente **App**, invoca o método **map()**. Este método faz o mapeamento de um *array* ou objeto de dados e gera código HTML. Aqui o método recebe uma função *callback* que retorna cada objeto **livro** existente em **props.livros**.

TabelaBody.js

```
import React from "react";
const TabelaBody = props => (
  <tbody>
    { props.livros.map((livro) => (
      <tr>
        <td>{ livro.id }</td>
        <td>{ livro.titulo }</td>
        <td>{ livro.autor }</td>
        <td>
          <button className="excluir">Excluir</button>
        </td>
      </tr>
    ))}
  </tbody>
);
export default TabelaBody;
```

Essa sintaxe insere o **id**, o **título** e o **autor** de cada livro nas respectivas células da tabela.

Tabela de Livros			
ISBN	Título	Autor	
9786589711612	Crime e Castigo	Fiódor Dostoiévski	<button>Excluir</button>
9788525056009	Admirável Mundo Novo	Aldous Huxley	<button>Excluir</button>
9788535909555	A Revolução dos Bichos	George Orwell	<button>Excluir</button>
Rodapé da Tabela			

Definindo o Estilo da Tabela

- Para definir um estilo para a tabela, é preciso incluir as regras CSS no arquivo **index.css** da aplicação.

```
body {
  font: 16px sans-serif;
  margin: 0;
}
.tabela {
  border-collapse: collapse;
  border: 1px solid #ccc;
  width: calc(100% - 20px);
  margin: 20px auto;
}
.tabela th,
.tabela td {
  padding: 10px 5px;
  border-bottom: 1px solid #ccc;
}
thead > tr:last-child {
  text-align: left;
}
thead > tr:first-child {
  background: #5bbdb6;
}
```

```
tbody > tr:nth-child(2n-1) {
  background: #f1f1f1;
}
tfoot > tr {
  text-align: center;
  background: #5bbdb6;
}
.botao {
  display: inline-block;
  font-weight: 400;
  color: #212529;
  text-align: center;
  vertical-align: middle;
  cursor: pointer;
  background-color: transparent;
  border: 1px solid transparent;
  padding: 0.375rem 0.75rem;
  font-size: 1rem;
  line-height: 1.5;
  border-radius: 0.25rem;
  transition: color 0.15s ease-in-out, background-color 0.15s ease-in-out,
    border-color 0.15s ease-in-out, box-shadow 0.15s ease-in-out;
}
```

```
.remover {
  color: #fff;
  background-color: #dc3545;
  border-color: #dc3545;
}
.remover:hover {
  color: #fff;
  background-color: #c82333;
  border-color: #bd2130;
}
.editar {
  color: #fff;
  background-color: #007bff;
  border-color: #007bff;
}
.editar:hover {
  color: #fff;
  background-color: #0069d9;
  border-color: #0062cc;
}
```

Definindo a Quantidade de Livros

- Também é necessário incluir a **quantidade de livros** no rodapé da tabela.

```
render() {  
  return (  
    <table className="tabela">  
      <TabelaHead />  
      <TabelaFoot qdeLivros = { this.state.livros.length } />  
      <TabelaBody livros={ this.state.livros } />  
    </table>  
  );  
}
```

App.js

A propriedade **length** retorna a quantidade de livros do *array livros*. Essa quantidade, por sua vez, será passada para o componente **TabelaFoot**, que exibirá a quantidade.

```
import React from "react";  
const TabelaFoot = props => (  
  <tfoot>  
    <tr>  
      <td colspan="4">Quantidade de Livros: { props.qdeLivros }</td>  
    </tr>  
  </tfoot>  
);  
export default TabelaFoot;
```

TabelaFoot.js

O componente **TabelaFoot** recebe a quantidade de livros, como o parâmetro **props** da função, e exibe esta quantidade no rodapé.

Tabela de Livros			
ISBN	Título	Autor	
9786589711612	Crime e Castigo	Fiódor Dostoiévski	<button>Excluir</button>
9788525056009	Admirável Mundo Novo	Aldous Huxley	<button>Excluir</button>
9788535909555	A Revolução dos Bichos	George Orwell	<button>Excluir</button>
Quantidade de Livros: 3			

Definindo uma Fonte de Dados

- Em uma aplicação real, os dados da tabela seriam recuperados de um **banco de dados** e não de um *array* inserido no componente **App**.
- Para simular essa situação, será pressuposto que o banco de dados da aplicação fornece uma API, a qual disponibiliza dados em **formato JSON**.
- Para simular os dados da API, crie a pasta **api** no diretório **public** do projeto. Depois, crie o arquivo **livros.json** na pasta **api** com o conteúdo ao lado.

*Para carregar as linhas da tabela de **forma dinâmica**, serão escritos scripts para **consulta aos dados** da API e para **organização dos valores recuperados** nas linhas da tabela. Para isso, será usada uma funcionalidade nativa dos navegadores denominada **API Fetch**.*

```
[  
  {  
    "id": "9786589711612",  
    "titulo": "Crime e Castigo",  
    "autor": "Fiódor Dostoiévski"  
  },  
  {  
    "id": "9788525056009",  
    "titulo": "Admirável Mundo Novo",  
    "autor": "Aldous Huxley"  
  },  
  {  
    "id": "9788535909555",  
    "titulo": "A Revolução dos Bichos",  
    "autor": "George Orwell"  
  }  
]
```

Definindo uma Fonte de Dados

- Para que a aplicação possa recuperar os dados do arquivo JSON, é preciso adequar o componente **App**.

Os dados do objeto **state** devem ser removidos.

```
class App extends Component {  
  state = {  
    livros: []  
  };  
  componentDidMount() {  
    fetch("/api/livros.json")  
      .then(response => response.json())  
      .then(livros => this.setState({ livros })))  
      .catch(function(error) {  
        console.log("Erro na requisição");  
      })  
      .finally(function() {  
        console.log("Sempre executa");  
      });  
  }  
  render() {
```

É preciso incluir o método **componentDidMount()**. Este método é chamado imediatamente após o método **render()**, ou seja, logo após ter ocorrido a renderização do componente.

O método **fetch(url)**, nativo do navegador, faz a consulta ao arquivo JSON e retorna uma *promise*.

Os métodos **then()** tentam resolver a *promise*. Ao resolvê-la, os dados do arquivo JSON são recebidos e, em seguida, objeto **state** é populado com estes dados.

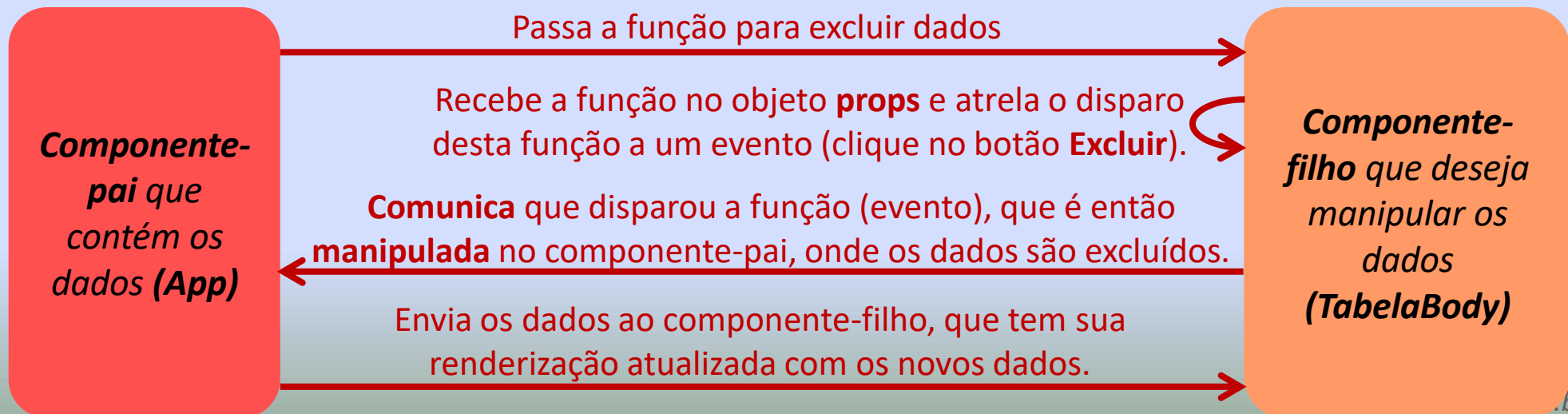
O método **catch()** faz o tratamento em caso de erro na requisição. Em uma aplicação real, poderia ser definido um script para renderizar uma mensagem de erro ao usuário.

O método **finally()** sempre é executado e deve ser implementado conforme a necessidade do projeto.

Ao renderizar a página, a tabela deverá ter a mesma aparência e os mesmos dados.

Implementando o Botão Excluir

- A seguir, será implementado o botão **Excluir**, para que, ao clicar nele, a linha correspondente na tabela seja apagada.
- Um dos fundamentos do React é que os dados só podem ser alterados pelo componente onde esses dados residem. Sendo assim, como é possível apagar uma linha da tabela que está no componente **TabelaBody** se os dados residem no componente **App**?
- O React torna isso possível por meio de um mecanismo que consiste em **comunicar** e **manipular** eventos. No caso da exclusão, ele funcionaria da seguinte forma:



Implementando o Botão Excluir

```
handleExcluirLinha = ( ) => console.log("Botão clicado"); App.js
render() {
  return (
    <table className="tabela">
      <TabelaHead />
      <TabelaFoot qdeLivros = { this.state.livros.length } />
      <TabelaBody
        livros={ this.state.livros }
        excluirLinha={ this.handleExcluirLinha }
      />
    </table>
  );
}
```

No componente **App**, que armazena os dados, deve ser criada a função **handleExcluirLinha**. Ela será executada quando o botão **Excluir** do componente **TabelaBody** for clicado.

Em seguida, é preciso passar a função criada para o componente **TabelaBody**, da mesma maneira que foi passado o *array livros* anteriormente.

```
{ props.livros.map((livro) => (
  <tr>
    <td>{ livro.id }</td>
    <td>{ livro.titulo }</td>
    <td>{ livro.autor }</td>
    <td>
      <button className="excluir"
        onClick={ ( ) => props.excluirLinha() }>Excluir</button>
    </td>
  </tr>
))}
```

TabelaBody.js

No componente **TabelaBody**, capture o clique do botão **Excluir** e atrole este evento à execução da função definida no componente **App** (passada para esse componente como **props**).

*Agora, ao clicar em um dos botões **Excluir**, deverá aparecer o texto "Botão clicado" no console do navegador, indicando que a comunicação entre os componentes foi estabelecida.*

Implementando o Botão Excluir

- É preciso que, ao clicar em um botão **Excluir**, o script identifique a linha correspondente e apague somente ela.
- As linhas da tabela podem ser identificadas pelo ISBN do livro, cujo nome no arquivo JSON é **id**. Esse id será usado como parâmetro da função de exclusão, para apagar a linha onde se encontra o botão clicado.

App.js

```
handleExcluirLinha = id => {  
  const livros = this.state.livros.filter(l => l.id !== id);  
  this.setState({ livros });  
};
```

Altere a função **handleExcluirLinha**, para que ela receba como parâmetro o **id** da linha que contém o botão clicado. O método **filter()** é aplicado no objeto **state** que contém os dados de todos os livros. O filtro cria um novo objeto contendo todos os livros, exceto aquele cujo id é o da linha onde o botão foi clicado. Esse objeto é definido como o novo objeto **state**, provocando a atualização da tabela sem a linha apagada.

TabelaBody.js

```
<button className="excluir"  
  onClick={() => props.excluirLinha(livro.id)}  
  id={ livro.id }>Excluir</button>
```

No botão **Excluir**, passe o **id** do livro, como parâmetro da função que apaga a linha.

*Agora, ao clicar em um dos botões **Excluir**, a linha onde se encontra o botão será apagada e a quantidade de linhas será atualizada no rodapé da tabela.*

Implementando a Ordenação da Tabela

- A última funcionalidade a ser implementada é a ordenação das linhas da tabela em ordem alfabética pelo **título** do livro.

```
<th>ISBN</th>
<th>Título
  <div className="container-setinhas">
    <div>&#129093;</div>
    <div>&#129095;</div>
  </div>
</th>
<th>Autor</th>
<th></th>
```

TabelaHead.js

É preciso modificar o componente **TabelaHead** para inserir duas setinhas ao lado da palavra **Título** no cabeçalho da tabela.

Esse container **<div>** cria o contexto para posicionar os dois elementos **<div>** filhos, ou seja, as setinhas (uma em cima da outra).

A setinha para cima é obtida com a codificação de caracteres **🡅**. E a setinha para baixo com a codificação **🡇**. Com isso, as setinhas serão renderizadas **embaixo** da palavra **Título**.

```
.container-setinhas {
  display: inline-block;
  width: 20px;
  height: 40px;
  margin-left: 5px;
  vertical-align: middle;
}
.container-setinhas > div { cursor: pointer; }
```

index.css

No arquivo **index.css**, insira essas duas regras CSS. A 1ª regra posiciona as setinhas **ao lado** da palavra **Título**. E a 2ª faz com que, ao passar o cursor do mouse sobre as setas, ele mude para *pointer*.

Implementando a Ordenação da Tabela

- Ao clicar na seta **para cima**, os livros serão ordenados pelo título em **ordem alfabética crescente**. E ao clicar na seta **para baixo**, a ordenação será em **ordem alfabética decrescente**. Para isso, será novamente

App.js aplicado o mecanismo do React de comunicar e manipular eventos.

```
handleOrdenarCrescente = titulo => {  
  const livros = this.state.livros.sort((a, b) =>  
    a.titulo < b.titulo ? -1 : 0  
  );  
  this.setState({ livros });  
};  
handleOrdenarDecrescente = titulo => {  
  const livros = this.state.livros.sort((a, b) =>  
    a.titulo < b.titulo ? -1 : 0  
  );  
  livros.reverse();  
  this.setState({ livros });  
};  
render() {  
  return (  
    <table className="tabela">  
      <TabelaHead  
        ordenarCrescente = { this.handleOrdenarCrescente }  
        ordenarDecrescente = { this.handleOrdenarDecrescente }  
      />
```

No componente **App**, logo após o fechamento da função **handleExcluirLinha**, inclua os seguintes manipuladores de eventos, que são as funções JavaScript para ordenar em ordem alfabética crescente e decrescente.

Passe os manipuladores de evento para o componente **TabelaHead** usando os atributos **ordenarCrescente** e **ordenarDecrescente**.

Implementando a Ordenação da Tabela

TabelaHead.js

```
const TabelaHead = props => (  
  <thead>  
    <tr>  
      <th colSpan="4">Tabela de Livros</th>  
    </tr>  
    <tr>  
      <th>ISBN</th>  
      <th>Título  
        <div className="container-setinhas">  
          <div onClick={ () => props.ordenarCrescente() }>#129093; </div>  
          <div onClick={ () => props.ordenarDecrescente() }>#129095;</div>  
        </div>  
      </th>  
    </tr>  
  </thead>  
)
```

No componente **TabelaHead**, capture o clique nas setinhas e atrele estes eventos à execução das funções definidas no componente **App** (passadas para esse componente como **props**).

Ao armazenar ou compartilhar um projeto React, é importante não manter o enorme diretório **node_modules** do projeto. Embora ele contenha todos os módulos e funcionalidades necessários para utilizar o React, este diretório pode ser facilmente recuperado rodando o comando **npm install** na pasta do projeto. Finalizada a instalação do diretório, basta executar o comando **npm start** para iniciar a aplicação.

Tabela de Livros			
ISBN	Título 	Autor	
9788535909555	A Revolução dos Bichos	George Orwell	<button>Excluir</button>
9788525056009	Admirável Mundo Novo	Aldous Huxley	<button>Excluir</button>
9786589711612	Crime e Castigo	Fiódor Dostoiévski	<button>Excluir</button>
Quantidade de Livros: 3			

Referências

- SILVA, Maurício S. **React: Aprenda Praticando**. São Paulo: Editora Novatec, 2021.
- TORRES, Fernando E. **Single Page Application**. São Paulo: Editora Senac, 2023.